

Tài liệu Bổ sung

Bài tập Lập trình

Canny · Multi-scale Hough · RANSAC

Tài liệu tự học · Tuần 7-8 · Học kỳ 2

Cách sử dụng tài liệu. Tài liệu này là hướng dẫn tự học bổ sung cho **ba bài tập lập trình**. Mỗi bài yêu cầu bạn tự cài đặt thuật toán từ đầu bằng NumPy – không sử dụng hàm OpenCV tương ứng ở phần cốt lõi – để nắm vững nguyên lý trước khi so sánh với kết quả thư viện. Không có mã nguồn trong tài liệu; phần cài đặt là nhiệm vụ của bạn.

Bài	Chủ đề	Kỹ thuật cốt lõi
1	Standard Canny	Gaussian, Sobel, NMS, Hysteresis, Otsu
2	Multi-scale Hough	Accumulator đa phân giải, top- k peaks, gradient-directed voting
3	RANSAC tùy chọn	Random sampling, consensus, fitting $s < 5$ tham số

1. Bài 1 - Cài đặt Canny từ Đầu

1.1. Tại sao Cạnh Quan trọng?

Khi nhìn vào một vật thể, não nhận diện nó chủ yếu qua **đường viền và hình dạng**, không phải qua từng giá trị màu sắc. Máy tính làm điều tương tự: **cạnh ảnh** là nơi cường độ pixel thay đổi đột ngột, mang thông tin về biên vật thể, chiều sâu, bóng sáng, và kết cấu bề mặt.

Câu hỏi đặt ra: làm thế nào để phát hiện những chỗ thay đổi đó một cách *chính xác và đáng tin cậy* trên ảnh có nhiều thực tế?

1.2. Gradient Ảnh - Building Block Cốt lõi

Nếu cạnh là nơi cường độ thay đổi đột ngột, thì cạnh tương ứng với nơi **đạo hàm của cường độ đạt giá trị lớn**. Trên ảnh rời rạc, đạo hàm được xấp xỉ bằng sai phân:

Đạo hàm rời rạc và Gradient

$$G_x(x, y) = I(x + 1, y) - I(x, y) \quad (\text{theo chiều } x)$$

$$G_y(x, y) = I(x, y + 1) - I(x, y) \quad (\text{theo chiều } y)$$

$$M(x, y) = \sqrt{G_x^2 + G_y^2} \quad (\text{gradient magnitude})$$

$$\theta(x, y) = \arctan\left(\frac{G_y}{G_x}\right) \quad (\text{gradient direction})$$

Lưu ý quan trọng. θ luôn **vuông góc với đường cạnh thật** - không phải song song với nó. Nếu cạnh chạy theo chiều dọc, gradient sẽ hướng ngang. Nhầm lẫn điều này sẽ làm sai bước NMS.

1.3. Bộ lọc Sobel

Sai phân đơn giản quá nhạy với nhiễu. Bộ lọc **Sobel** kết hợp lấy đạo hàm theo một chiều với làm mịn (smoothing) theo chiều vuông góc bằng Gaussian 1D:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * I \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * I$$

Các trọng số 2 ở hàng/cột giữa tương ứng với phần smoothing Gaussian.

Ba lỗi phổ biến khi implement lần đầu.

1. Tính gradient trực tiếp trên ảnh gốc chưa qua blur - nhiều tạo gradient giả lan tràn.
2. Dùng $|G_x| + |G_y|$ thay vì $\sqrt{G_x^2 + G_y^2}$ - gây sai hướng θ , NMS sẽ chọn nhầm hàng xóm.
3. Quên chuyển sang grayscale - gradient trên 3 kênh RGB không nhất quán với nhau.

1.4. Pipeline Canny - Bốn Bước

Nguyên tắc thiết kế: mỗi bước giải quyết đúng một vấn đề cụ thể.

Bước	Thao tác	Vấn đề giải quyết
1	Gaussian blur (σ)	Nhiều tạo gradient giả
2	Sobel $\rightarrow M, \theta$	Cần biết độ mạnh và hướng cạnh
3	Non-maximum Suppression	Cạnh dày 3-5 px $\rightarrow$ mỏng 1 px
4	Hysteresis Thresholding	Phân biệt cạnh thật và nhiễu còn sót

1.4.1. Bước 1 - Gaussian Smoothing

Làm mờ ảnh trước khi tính gradient. σ nhỏ giữ chi tiết nhưng nhạy nhiễu; σ lớn lọc nhiễu tốt nhưng mờ chi tiết nhỏ. Điểm khởi đầu thực tế: $\sigma = 1.0$ đến 2.0.

1.4.2. Bước 2 - Gradient M và θ

Áp dụng Sobel **lên ảnh đã blur** để ra G_x, G_y , tính M và θ . Thứ tự quan trọng: blur trước, Sobel sau.

Ví dụ tính tay - một pixel tại (2, 2).

Giả sử sau blur, patch 3×3 xung quanh pixel là:

$$\begin{bmatrix} 100 & 105 & 110 \\ 98 & 106 & 115 \\ 95 & 100 & 108 \end{bmatrix}$$

Áp dụng G_x kernel (cột phải trừ cột trái, trọng số 1-2-1):

$$G_x = (110 + 2 \times 115 + 108) - (100 + 2 \times 98 + 95) = 448 - 391 = 57$$

Áp dụng G_y kernel (hàng dưới trừ hàng trên):

$$G_y = (95 + 2 \times 100 + 108) - (100 + 2 \times 105 + 110) = 403 - 420 = -17$$

$M = \sqrt{57^2 + (-17)^2} = \sqrt{3538} \approx 59.5 \quad \theta = \arctan(-17/57) \approx -16.6^\circ$
 Gradient hướng hơi xuống-phải; cạnh ở đây nghiêng hơi lên-trái.

1.4.3. Bước 3 - Non-maximum Suppression (NMS)

Giữ lại chỉ pixel có M là **cực đại cục bộ** dọc theo hướng gradient θ .

Thuật toán cho mỗi pixel p :

1. Lượng hoá $\theta(p)$ về 1 trong 4 hướng: $0^\circ, 45^\circ, 90^\circ, 135^\circ$ (mỗi vùng rộng 45° , xử lý wrap-around trên $[0^\circ, 180^\circ)$).
2. Xác định hai hàng xóm q_1, q_2 dọc theo hướng đó.
3. Nếu $M(p) < M(q_1)$ hoặc $M(p) < M(q_2)$: **xoá** p (gán 0).
4. Ngược lại: **giữ** p .

Ví dụ NMS - patch 3×3 với $\theta = 90^\circ$.

$$M = \begin{bmatrix} \cdot & 8 & \cdot \\ \cdot & \mathbf{80} & \cdot \\ \cdot & 60 & \cdot \end{bmatrix}$$

$\theta = 90^\circ \Rightarrow$ so sánh với hàng xóm **trên** ($q_2 = 8$) và **dưới** ($q_1 = 60$):

$$80 \geq 8 \checkmark \quad 80 \geq 60 \checkmark \quad \Rightarrow \quad \mathbf{GIỮ LẠI}$$

Nếu $M(p) = 50$: $50 < 60 \Rightarrow$ **LOẠI BỎ**.

1.4.4. Bước 4 - Hysteresis Thresholding

Dùng **hai ngưỡng** $\tau_{\text{low}} < \tau_{\text{high}}$:

- $M(p) \geq \tau_{\text{high}}$: **strong edge** - giữ chắc chắn.
- $\tau_{\text{low}} \leq M(p) < \tau_{\text{high}}$: **weak edge** - giữ nếu kết nối với strong.
- $M(p) < \tau_{\text{low}}$: **nhieũ** - loại bỏ.

Edge tracking: từ mỗi strong edge, duyệt 8-kết nối (DFS/BFS) để tìm tất cả weak edge có thể đến được. Weak edge không kết nối với bất kỳ strong edge nào bị loại bỏ.

Ví dụ Edge Tracking - chuỗi $S \rightarrow W1 \rightarrow W2, W3$ cô lập.

$$S_{(M=120)} \longrightarrow W_{1(M=55)} \longrightarrow W_{2(M=48)} \\ W_{3(M=52)} \text{ (không kể } S, W1, \text{ hay } W2)$$

$$\tau_{\text{low}} = 40, \tau_{\text{high}} = 100.$$

- S là strong $\rightarrow$ bắt đầu DFS.

- W_1 kể $S \rightarrow$ **giữ lại**, thêm vào stack.
- W_2 kể W_1 (đã promote) $\rightarrow$ **giữ lại**.
- W_3 không kể strong nào $\rightarrow$ **loại**.

Lỗi hay gặp với single-pass hysteresis. Nếu implement hysteresis bằng một vòng lặp đơn, bạn sẽ bỏ sót chuỗi $W_1 \rightarrow W_2 \rightarrow W_3$ vì W_2 được xét trước khi W_1 được promote. DFS/BFS lan truyền đúng theo thứ tự.

1.5. Tự động chọn Ngưỡng - Otsu Thresholding

Chỉnh τ_{low} và τ_{high} bằng tay cho từng ảnh tốn thời gian. **Phương pháp Otsu** tự động tìm ngưỡng tối ưu từ histogram của bản đồ gradient M .

Ý tưởng Otsu: chọn ngưỡng τ^* tối đa hoá **phương sai liên lớp** (between-class variance):

$$\tau^* = \arg \max_{\tau} \omega_0(\tau) \omega_1(\tau) [\mu_0(\tau) - \mu_1(\tau)]^2$$

trong đó ω_k là tỷ lệ pixel trong lớp k và μ_k là trung bình cường độ lớp k .

Áp dụng Otsu cho Canny.

Histogram của bản đồ M thường có dạng bimodal:

- Đỉnh trái ở giá trị thấp - pixel vùng phẳng, không phải cạnh.
- Đỉnh phải ở giá trị cao hơn - pixel cạnh thật.

Otsu tìm τ^* tại đáy thung lũng giữa hai đỉnh. Dùng trong Canny:

$$\tau_{\text{high}} = \tau_{\text{Otsu}}^*, \quad \tau_{\text{low}} = \tau_{\text{high}}/2$$

Tỷ lệ 1 : 2 giữa hai ngưỡng là điểm khởi đầu thực tế, có thể fine-tune tùy ảnh.

Khi nào Otsu hoạt động tốt / khi nào không?

Otsu tối ưu khi histogram **thực sự bimodal** - ảnh có vùng cạnh rõ và vùng phẳng rõ ràng. Khi ảnh có kết cấu dày đặc (vải, cỏ), histogram gần unimodal và Otsu trả về ngưỡng tùy tiện. Luôn visualise histogram trước khi tin vào kết quả Otsu.

1.6. Yêu cầu Bài tập 1

1. Cài đặt toàn bộ pipeline Canny từ đầu bằng NumPy (không dùng **cv2.Canny** ở phần cốt lõi).
2. Cài đặt Otsu để tự động xác định τ_{high} ; so sánh kết quả với τ chỉnh tay - khi nào Otsu tốt hơn, khi nào tệ hơn? Giải thích.

3. So sánh implementation của bạn với `cv2.Canny` bằng chỉ số **IoU** (Intersection over Union).
4. Trả lời: nếu kết quả có quá nhiều cạnh giả, bạn điều chỉnh tham số nào trước tiên và theo hướng nào? Giải thích dựa trên cơ chế thuật toán.

2. Bài 2 - Multi-scale Hough Transform

2.1. Nền tảng - Hough Transform Chuẩn

2.1.1. Tại sao không dùng $y = mx + b$?

Phương trình $y = mx + b$ không biểu diễn được đường thẳng đứng vì $m \rightarrow \infty$. Biểu diễn **Normal Form** giải quyết điều này:

$$\rho = x \cos \theta + y \sin \theta$$

trong đó ρ là khoảng cách từ gốc tọa độ đến đường thẳng và θ là góc của pháp tuyến. Phạm vi: $\rho \in [-d_{\max}, +d_{\max}]$, $\theta \in [0^\circ, 180^\circ)$ với $d_{\max} = \sqrt{W^2 + H^2}$. Mọi đường thẳng - kể cả thẳng đứng - đều có biểu diễn hữu hạn.

2.1.2. Insight Cốt lõi: Điểm $\rightarrow$ Đường cong Sine

Với điểm ảnh (x_0, y_0) cố định, phương trình $\rho = x_0 \cos \theta + y_0 \sin \theta$ là một **đường cong sine** trong không gian (ρ, θ) .

Insight: hai điểm cùng nằm trên một đường thẳng $L \Leftrightarrow$ hai đường cong sine tương ứng **cắt nhau** tại (ρ^*, θ^*) - tham số của L .

Thuật toán bỏ phiếu:

1. Khởi tạo accumulator $A[\rho][\theta] = 0$.
2. Với mỗi pixel cạnh (x_i, y_i) : với mọi θ_k , tính $\rho_k = x_i \cos \theta_k + y_i \sin \theta_k$, tăng $A[\rho_k][\theta_k] += 1$.
3. Tìm local maxima trong A - đó là các đường thẳng.

Ví dụ tính tay - ba điểm $(1, 1)$, $(2, 2)$, $(3, 3)$ trên đường $y = x$.

Đường $y = x$ có biểu diễn $\rho = 0$, $\theta = 135^\circ$ (vì $\cos 135^\circ = -0.707$, $\sin 135^\circ = 0.707$):

$$\rho_{(1,1)} = 1 \times (-0.707) + 1 \times 0.707 = 0 \quad \checkmark$$

$$\rho_{(2,2)} = 2 \times (-0.707) + 2 \times 0.707 = 0 \quad \checkmark$$

$$\rho_{(3,3)} = 3 \times (-0.707) + 3 \times 0.707 = 0 \quad \checkmark$$

Ba điểm đều vote vào ô $A[0][135^\circ] \Rightarrow$ ô này là đỉnh cao nhất $\Rightarrow$ phát hiện $y = x$.

2.2. Vấn đề của Hough Chuẩn

Hough chuẩn dùng một cặp $(\Delta\rho, \Delta\theta)$ cố định. Điều này gây ra ba vấn đề:

- **Đường thẳng song song gần nhau:** hai đường cách nhau 5 pixel tạo hai đỉnh accumulator cách nhau $\Delta\rho_{\text{peak}} = 5$. Nếu $\Delta\rho = 8$, hai đỉnh gộp thành một - phát hiện một đường khi thực ra có hai.
- **Chi phí không cân xứng:** phân giải mịn ($\Delta\rho = 0.5, \Delta\theta = 0.25^\circ$) cho accumulator rất lớn và chậm, dù chỉ cần thiết cho trường hợp đặc biệt.
- **Mismatch theo độ dài đường:** đường dài tạo đỉnh nhọn, đường ngắn tạo đỉnh rộng mờ. Một $(\Delta\rho, \Delta\theta)$ cố định không phù hợp tốt cho cả hai.

2.3. Multi-scale Hough - Hai Ý tưởng Kết hợp

2.3.1. Ý tưởng 1: Nhiều mức phân giải (Coarse-to-Fine)

Xây accumulator ở nhiều mức $(\Delta\rho, \Delta\theta)$ khác nhau:

Mức	$\Delta\rho$	$\Delta\theta$	Phù hợp cho
Coarse	8 px	4°	Đường dài, tìm vùng ứng viên
Medium	2 px	1°	Đa số ứng dụng
Fine	0.5 px	0.25°	Đường ngắn, song song gần nhau

Chiến lược coarse-to-fine: chạy accumulator coarse để tìm vùng (ρ, θ) có đỉnh cao. Chỉ zoom vào những vùng đó bằng accumulator fine. Kết quả: tốc độ gần bằng coarse, độ chính xác gần bằng fine.

Ý nghĩa Laplacian Scale.

Định nghĩa **Laplacian của accumulator** tại mức l :

$$\mathcal{L}_l = A_l - \text{upsample}(A_{l+1})$$

$\mathcal{L}_l$ bắt cấu trúc có ở mức l nhưng không có ở mức thô hơn $l+1$ - tức là đường thẳng mà độ phân giải tự nhiên là mức l .

Tương tự SIFT: keypoint được detect ở mức scale mà Laplacian response cực đại. Ở đây: đường thẳng được detect ở mức $(\Delta\rho, \Delta\theta)$ tối ưu cho độ dài của nó.

2.3.2. Ý tưởng 2: Top- k Peaks thay vì Threshold cố định

Thay vì đặt một threshold cố định cho accumulator, tìm tất cả **local maxima** và giữ lại k đỉnh cao nhất.

Thuật toán top- k peaks:

1. Áp dụng Gaussian blur nhẹ lên accumulator để làm mịn nhiễu lượng tử hoá.
2. Tìm local maxima: ô $A[\rho_0][\theta_0]$ là local max nếu nó lớn hơn tất cả ô lân cận trong vùng suppression radius r .

3. Sắp xếp các local maxima theo giá trị vote giảm dần.
4. Giữ k đỉnh đầu tiên.

Tại sao top- k tốt hơn threshold cố định?

Threshold cố định không chuyển được giữa các ảnh: ảnh 4K tích lũy nhiều vote hơn ảnh 200×200 cho cùng một đường thẳng.

Top- k đặt câu hỏi “có bao nhiêu đường thẳng?” thay vì “đỉnh nào đủ cao?” – thường dễ xác định hơn. Ví dụ: trong bài toán lane detection, luôn tìm đúng 2 đường (trái và phải).

NMS trên Accumulator. Sau khi tìm local maxima, áp dụng Non-maximum Suppression trong không gian (ρ, θ) với suppression radius phù hợp (ví dụ: 10 bước ρ và 10 bước θ). Điều này ngăn một đường thẳng thật được đếm nhiều lần do đỉnh accumulator bị trải rộng vì nhiễu lượng tử hoá.

2.3.3. Ý tưởng 3: Gradient-Directed Windowed Voting

Trong Hough chuẩn, mỗi pixel cạnh vote cho **tất cả 180 giá trị** θ . Nhưng hướng gradient θ_{actual} tại pixel đó *đã cho biết* θ của đường thẳng thật phải gần θ_{actual} . Tại sao phải vote cho các θ xa?

Windowed voting: chỉ vote cho θ_k nằm trong cửa sổ $\pm$ window quanh θ_{actual} :

$$\delta = \min(|\theta_{\text{actual}} - \theta_k|, 180^\circ - |\theta_{\text{actual}} - \theta_k|) \quad (\text{cyclic distance trên } [0^\circ, 180^\circ))$$

Chỉ vote nếu $\delta \leq \text{window}$.

Ví dụ với window = $\pm 15^\circ$.

Pixel có $\theta_{\text{actual}} = 45^\circ$. Chỉ vote cho $\theta_k \in [30^\circ, 60^\circ]$ – 30 ô thay vì 180.

Approach	Vote/pixel	Tốc độ	Robust với nhiễu
Full vote (chuẩn)	180	Chậm nhất	Cao nhất
Windowed ($\pm 20^\circ$)	≈ 40	Nhanh 4.5×	Trung bình
Single vote	1	Nhanh nhất	Thấp nhất

Với window = 20–30°, tốc độ tăng đáng kể mà chất lượng giảm nhỏ.

Window size tối ưu. Thực nghiệm: chất lượng (F1) ổn định khi giảm window từ 90° xuống $\approx 20^\circ$, sau đó giảm nhanh. “Elbow” ở 20–30° là điểm hoạt động thực tế.

2.4. Yêu cầu Bài tập 2

1. Xây accumulator Hough từ đầu với ít nhất hai mức phân giải (coarse và fine).
2. Cài đặt coarse-to-fine: tìm ứng viên ở coarse, refine ở fine.
3. Cài đặt top- k peaks với NMS trên accumulator. So sánh kết quả khi $k = 1, 3, 5$ trên ảnh bàn cờ - k nào phù hợp nhất và tại sao?
4. Cài đặt gradient-directed windowed voting. Vẽ đồ thị F1 vs window size (từ 5° đến 90°). Tìm và giải thích điểm “elbow”.

3. Bài 3 – RANSAC với Hình dạng Tuỳ chọn

3.1. Tại sao Cần RANSAC?

Least Squares tối thiểu hoá tổng bình phương sai số – tối ưu khi dữ liệu chỉ có nhiễu Gaussian. Tuy nhiên, dữ liệu ảnh thực tế thường có **outlier**: điểm từ kết cấu nền, bóng, phản chiếu, hoặc vật thể bị che khuất.

Tại sao Least Squares gây với outlier?

Sai số bình phương $(y_i - f(x_i))^2$ khuếch đại ảnh hưởng của outlier:

Một outlier cách xa 10 đơn vị: $10^2 = 100$

100 inlier, mỗi điểm cách xa 1 đơn vị: $100 \times 1^2 = 100$

Một outlier duy nhất ảnh hưởng ngang bằng 100 inlier. Đường fit bị kéo mạnh về phía outlier.

RANSAC (Random Sample Consensus) – Fischler & Bolles, 1981 – giải quyết vấn đề này bằng cách chỉ fit trên sample nhỏ toàn inlier, không bao giờ để outlier tham gia fit.

3.2. Thuật toán RANSAC

Pipeline RANSAC lặp lại N lần:

1. **Sample**: chọn ngẫu nhiên s điểm từ tập dữ liệu (s = số điểm tối thiểu để xác định mô hình).
2. **Fit**: tính tham số mô hình từ s điểm đó.
3. **Count inliers**: đếm tất cả điểm có khoảng cách đến mô hình $< \epsilon$.
4. **Cập nhật**: nếu số inlier lần này $> \text{best_score}$, lưu lại mô hình này.
5. **Refit**: sau N iteration, fit lại trên toàn bộ consensus set (tất cả inlier của best_model).

Ví dụ - Fitting đường thẳng ($s = 2$).

Dữ liệu: 20 inlier trên $y = 1.8x + 0.5$ (nhiều $\sigma = 0.25$) + 8 outlier. $\epsilon = 0.6$, $N = 80$.

Iteration 1 (sample tốt): chọn 2 điểm inlier $\Rightarrow$ fit $m = 1.81, b = 0.48 \Rightarrow$ 19/28 điểm là inlier $\Rightarrow \text{best_score} = 19$.

Iteration 2 (sample xấu): chọn 1 inlier + 1 outlier $\Rightarrow$ fit lệch $m = 0.55, b = 3.2 \Rightarrow$ 4/28 điểm là inlier $\Rightarrow \text{best_score}$ không cập nhật.

Sau 80 iteration: $\text{best_model} \approx (m = 1.80, b = 0.50)$. ✓

3.3. Số Iteration N Cần Thiết

Bao nhiêu iteration để đảm bảo ít nhất một lần sample toàn inlier với xác suất p ?

$$N = \frac{\log(1 - p)}{\log(1 - (1 - e)^s)}$$

trong đó e là tỷ lệ outlier ước lượng, s là kích thước sample, p là xác suất thành công mong muốn (thường $p = 0.99$).

e	$s = 2$ (đường thẳng)	$s = 4$ (homography)
10%	5	7
30%	11	26
50%	17	72
70%	55	620

Số iteration N với $p = 0.99$.

Với đường thẳng ($s = 2$) và 50% outlier: chỉ cần 17 iteration. Đây là lý do RANSAC được dùng trong real-time systems.

3.4. Chọn ε

ε nên phản ánh **mức nhiễu đo lường thực tế**, không phải được rút ra từ dữ liệu. Với pixel-level edge detection: Canny localise cạnh đến ± 1 -2 pixel, nên $\varepsilon = 1.5$ -2.5 pixel là điểm khởi đầu hợp lý.

ε quá nhỏ: inlier thật bị phân loại là outlier, consensus set nhỏ, fit không ổn định.

ε quá lớn: outlier lọt vào consensus, fit bị kéo lệch.

3.5. Điều kiện Hình dạng Tuỳ chọn: $s < 5$

Bài tập yêu cầu bạn chọn **hai hình dạng tham số** với số tham số $s < 5$. Điều kiện này đảm bảo N iteration vẫn thực tế (với $s = 4$, $e = 50\%$: $N = 72$).

Quan trọng: bạn phải giải thích lý do chọn hình dạng. Không chỉ mô tả hình dạng - hãy trả lời: hình dạng này xuất hiện tự nhiên ở đâu trong thực tế? Tại sao RANSAC phù hợp hơn Hough hoặc Least Squares cho bài toán đó? Câu giải thích có trọng số ngang phần cài đặt.

Một số ví dụ hình dạng phù hợp:

Hình dạng	s	Ứng dụng điển hình
Đường thẳng 2D	2	Lane detection – quen thuộc, ít điểm độc đáo
Đường tròn 2D	3	Đồng xu, tế bào, mắt trong ảnh khuôn mặt
Ellipse 2D	4	Biển số xe, vật thể bầu dục trong ảnh y tế
Parabola	3	Quỹ đạo ném vật, cầu vồng, đường dây điện
Sine wave (A, ω, ϕ)	3	Tín hiệu tuần hoàn trong spectrogram
Superellipse	3	Hình dạng thiết kế công nghiệp
Homography (4 điểm)	4	Image stitching, AR – bài toán khó, điểm cao hơn

Điểm thưởng. Hình dạng độc đáo chưa có bạn nào trong lớp chọn sẽ được cộng điểm. Tính độc đáo được xét dựa trên *lý giải* – không chỉ dựa trên hình dạng lạ.

3.6. Cách Fit từ s Điểm – Ví dụ Đường Tròn ($s = 3$)

Phần khó nhất của bài tập là tự cài đặt hàm fit từ đúng s điểm.

Fit đường tròn $(x - a)^2 + (y - b)^2 = r^2$ **từ ba điểm.**

Mở rộng phương trình: $x_i^2 - 2ax_i + a^2 + y_i^2 - 2by_i + b^2 = r^2$.

Đặt $c = a^2 + b^2 - r^2$, viết lại thành hệ tuyến tính:

$$\underbrace{\begin{bmatrix} -2x_0 & -2y_0 & 1 \\ -2x_1 & -2y_1 & 1 \\ -2x_2 & -2y_2 & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} a \\ b \\ c \end{bmatrix}}_d = \underbrace{\begin{bmatrix} -(x_0^2 + y_0^2) \\ -(x_1^2 + y_1^2) \\ -(x_2^2 + y_2^2) \end{bmatrix}}_d$$

Giải bằng `numpy.linalg.solve(A, d)` $\rightarrow (a, b, c)$, sau đó $r = \sqrt{a^2 + b^2 - c}$.

Hàm distance:

$$\text{dist}(x_i, y_i, a, b, r) = \left| \sqrt{(x_i - a)^2 + (y_i - b)^2} - r \right|$$

3.7. Yêu cầu Bài tập 3

1. Chọn hai hình dạng tham số với $s < 5$. Viết phần giải thích lý do chọn – ít nhất 100 từ mỗi hình dạng.
2. Cài đặt RANSAC tổng quát nhận vào `fit_fn(points)` và `distance_fn(model, points)`. Dùng lại hàm đó cho cả hai hình dạng.
3. Tạo dữ liệu tổng hợp: inlier với nhiễu Gaussian σ nhỏ cộng outlier ngẫu nhiên

nhiên. Thử tỷ lệ outlier 20%, 40%, 60%.

4. Tính số iteration theo công thức, kiểm tra với thực nghiệm: với N nhỏ hơn công thức, kết quả có còn đúng không? Với bao nhiêu phần trăm các lần chạy thì kết quả vẫn đúng?